Mauricio Salinas

Backend Developer · Python · FastAPI · Django/DRF · Event-driven · AI agent tooling

Santiago, Chile · maundrex@gmail.com
mauridev94.github.io · github.com/MauriDev94 · linkedin.com/in/mauricio-salinas-rubio
Open to remote work · English C2

SUMMARY

Python backend developer. I build REST APIs and event-driven systems with **FastAPI** and **Django/DRF** using Clean Architecture, DDD and TDD: **700+ tests** across my three main projects, with integration tests against real PostgreSQL in CI and decisions recorded in ADRs. I built a **harness** that won't let AI coding agents call a task done until the tests are green.

EXPERIENCE

IT Support Analyst · Dominó (Cadaques) — Santiago, Chile · Apr 2023 – Jul 2024

- Built a **Python (pandas) ETL on SQL Server** that consolidated the **sales receipts issued in 2022–2023** by the holding's companies: millions of documents from a large branch network, collected, transformed and loaded automatically.
- Analyzed and resolved production incidents down to their root cause, escalating defects to the development team and following them through to resolution.
- Maintained production SQL databases, built reports for operations and automated repetitive tasks with Python scripts.

PROJECTS

Independent development · Apr 2025 – present

Event-Driven Orders · Microservices with RabbitMQ

> `3 microservices · 138 tests + e2e · 85–92% coverage per service · 19 ADRs`

Stack: Python 3.12 · FastAPI · RabbitMQ (aio-pika) · PostgreSQL 16 · Docker Compose · testcontainers · structlog · GitHub Actions

E-commerce orders handled by three services (orders, inventory and notifications) that communicate only through events.

- **Idempotent consumers**: a redelivered event never duplicates effects (`INSERT … ON CONFLICT DO NOTHING`).
- **Race-free stock reservation** (conditional UPDATE + row-level locking), tested against real PostgreSQL.
- **DLQ and retries with backoff** in 3 stages (transient vs. permanent failures); eventual consistency through choreography.
- **Observability**: JSON logs with an end-to-end correlation ID (HTTP → events → 3 services).

Repo: **github.com/MauriDev94/event-driven-orders**

MedBook API · Medical appointment booking with Django + DRF

> `305 tests · 99.4% branch coverage · TDD`

Stack: Python 3.12 · Django 5.2 LTS · DRF 3.18 · PostgreSQL 16 · simplejwt · drf-spectacular · pytest · factory-boy · Docker · GitHub Actions

Doctors publish schedules, the system generates slots and patients book them; each appointment follows a state machine (pending → confirmed → completed / cancelled / no-show).

- **RBAC** with 8 permission classes and object-level access control (doctor / patient / admin).
- **Safe concurrent booking** (transaction + atomic UPDATE), backed by a test where two threads race for one slot.
- No N+1 queries (`select_related` / `prefetch_related`), OpenAPI 3 with Swagger and login rate limiting.
- **TDD** (Red-Green-Refactor); CI runs lint, tests against real PostgreSQL and a 95% coverage gate.

Repo: **github.com/MauriDev94/MedBook** · Swagger: **medbook-oiio.onrender.com/api/docs** (first load ~50 s)

Monolith API · Modular monolith with FastAPI

> `263 tests · 85% branch-coverage gate · deployed on Render`

Stack: Python 3.12 · FastAPI · SQLAlchemy 2.0 · Alembic · PostgreSQL 16 · PyJWT · Argon2 · Docker · GitHub Actions

REST API with Clean Architecture and DDD across three modules (auth, users, todos) and its own React + TS frontend.

- **Full auth**: JWT with refresh-token rotation and reuse detection, Google SSO with CSRF protection (signed `state` cookie), OTPs stored as HMAC-SHA256, Argon2 and per-IP rate limiting.
- Framework-free domain; modules talk through ports (`UserProvider`) rather than shared models.
- Tests against real PostgreSQL with the actual Alembic migrations, plus an `alembic check` gate in CI that catches model/migration drift.

Repo: **github.com/MauriDev94/Api_monolith** · Swagger: **api-monolith.onrender.com/docs** (first load ~50 s) ·
Frontend: **monolith-frontend-delta.vercel.app**

MauriDev Harness · Deterministic verification for AI coding agents

Private repo

> `Claude Code + OpenCode · 3 stacks · used on 3 projects`

Stack: Python (standard library only) · TypeScript · Claude Code hooks · OpenCode plugin · ruff · mypy · pytest

An agent saying "done, tests pass" makes a claim, not a proof: this harness demands an exit code to back it up.

- **Deterministic core**: detects the stack and runs ruff, mypy, pytest and coverage in order; the exit code is the verdict.
- **Thin adapters** for Claude Code (`PostToolUse` and `Stop` hooks) and OpenCode (TypeScript plugin) that block edits breaking the gate and any "done" without evidence.
- Project scaffolding for Python, FastAPI and Django that starts green, plus an idempotent installer.

Other projects

- **Tag_Bot**: downloads highway toll receipts from Gmail automatically (Gmail API + Playwright scraping) and delivers them via Telegram. Private repo.

TECHNICAL SKILLS

Languages	Python, SQL, TypeScript
Backend	FastAPI, Django, Django REST Framework, Pydantic v2, SQLAlchemy 2.0, Alembic
Data	PostgreSQL, SQL Server, pandas, RabbitMQ (aio-pika), Supabase
Architecture	Clean Architecture, DDD, modular monolith, event-driven microservices, ADRs
Testing	pytest, TDD, testcontainers, factory-boy, branch coverage, Vitest
Security	JWT (rotation and reuse detection), OAuth2 / Google SSO, OTP, Argon2, RBAC, rate limiting
DevOps	Docker, Docker Compose, GitHub Actions, Render, Vercel, structlog (JSON logs), correlation IDs
Applied AI	Claude Code (skills, subagents, hooks), OpenCode, deterministic verification for agents
Practices	SOLID, Spec-Driven Development, Conventional Commits, pull requests and code review

LANGUAGES

Spanish: Native
English: C2 (EF SET 71/100)

EDUCATION

Programmer Analyst (Analista Programador) · Graduated
Instituto Profesional Santo Tomás · Santiago, Chile · Mar 2021 – Jul 2025